Coms 472/572 Recitation

HEURISTIC(WEEK-3)

Xuye Wu, Computer Science department

Example

A basic wooden railway set contains the pieces shown in Figure 3.32. The task is to connect these pieces into a railway that has no overlapping tracks and no loose ends where a train could run off onto the floor.

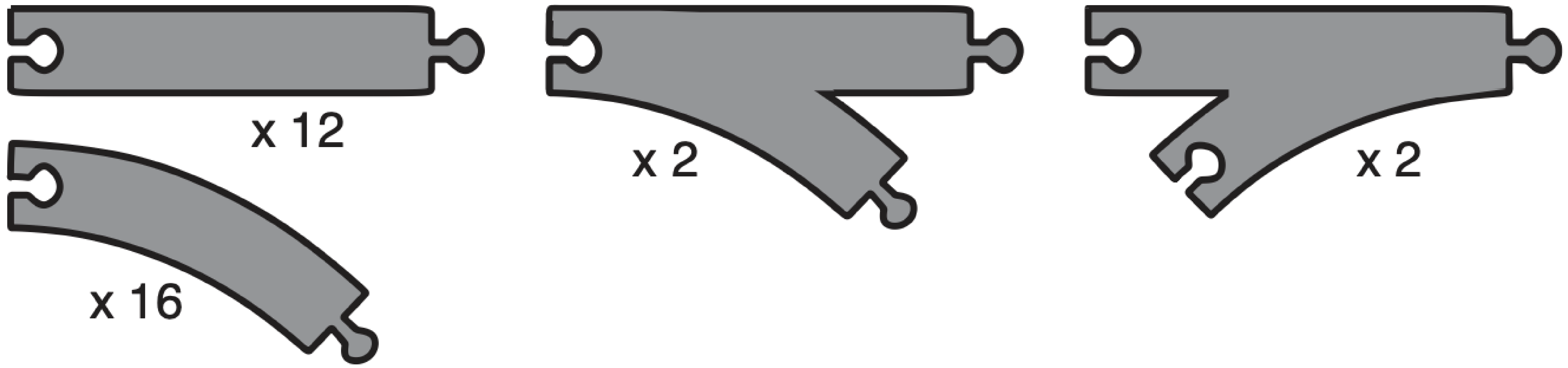


Figure 3.32 The track pieces in a wooden railway set; each is labeled with the number of copies in the set. Note that curved pieces and "fork" pieces ("switches" or "points") can be flipped over so they can curve in either direction. Each curve subtends 45 degrees.

Example

a. Suppose that the pieces fit together exactly with no slack. Give a precise formulation of the task as a search problem.

b. Identify a suitable uninformed search algorithm for this task and explain your choice.

c. Explain why removing any one of the “fork” pieces makes the problem unsolvable.

Example

a. Suppose that the pieces fit together exactly with no slack. Give a precise formulation of the task as a search problem.

States: States can be described as the current layout of the track.

Initial State: No pieces placed, all 32 available.

Actions: Place a piece (straight, curve, fork) at an open connector.

Transition Model: Updates open ends and reduces unused pieces.

Goal State: All pieces used, no open ends, no overlaps (closed loops).

Example

b. Identify a suitable uninformed search algorithm for this task and explain your choice.

Breadth-first search (BFS)?
BFS would expand level by level and store every partial layout in memory, which explodes combinatorially.

Iterative deepening?
iterative deepening would just redo partial expansions at shallower depths for no benefit.

DFS?

A solution requires placing all pieces (n = 32 total pieces). All solutions lie at the same depth at n. That means DFS can go directly down to depth n without worrying about missing a shorter solution.

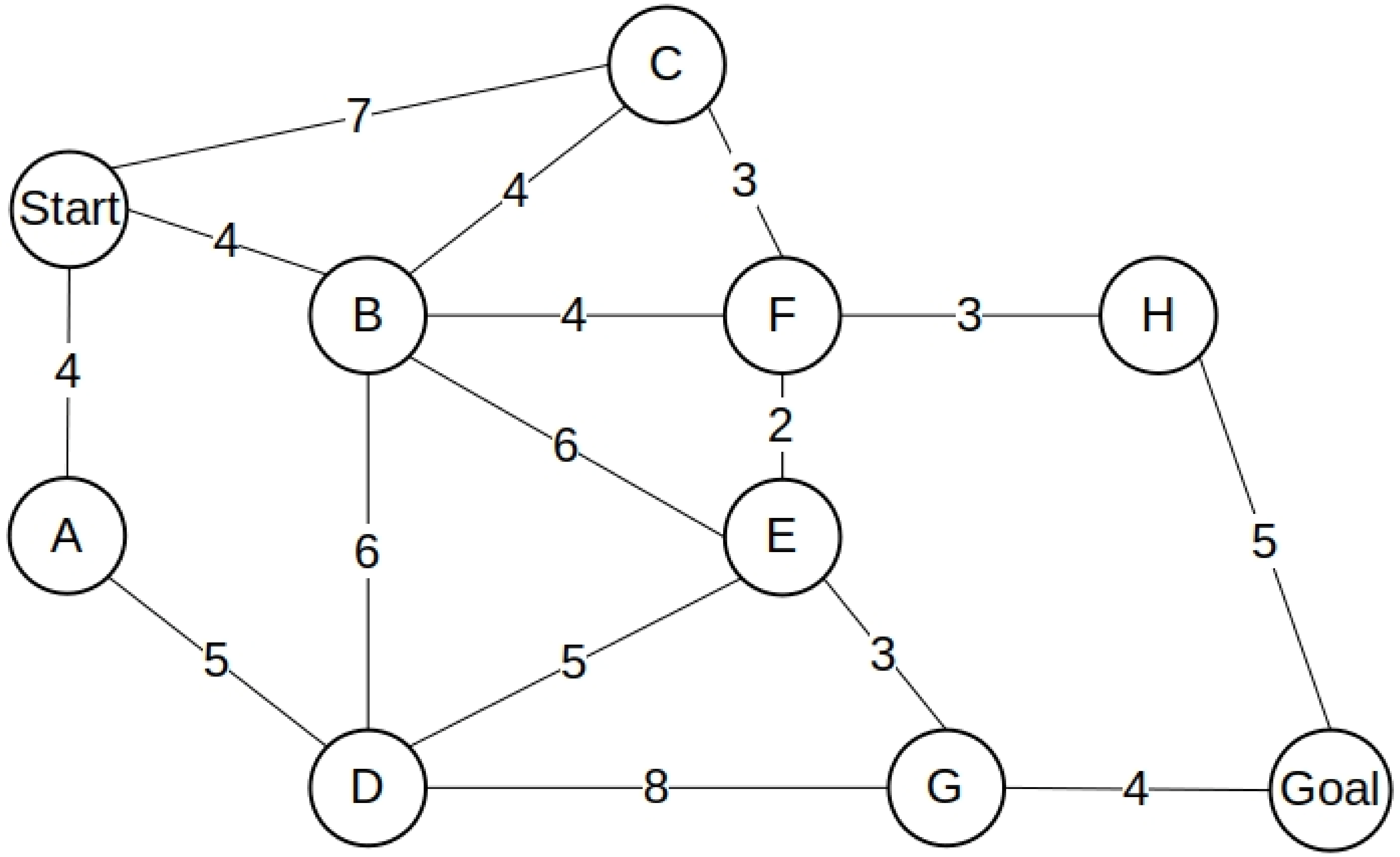
Example

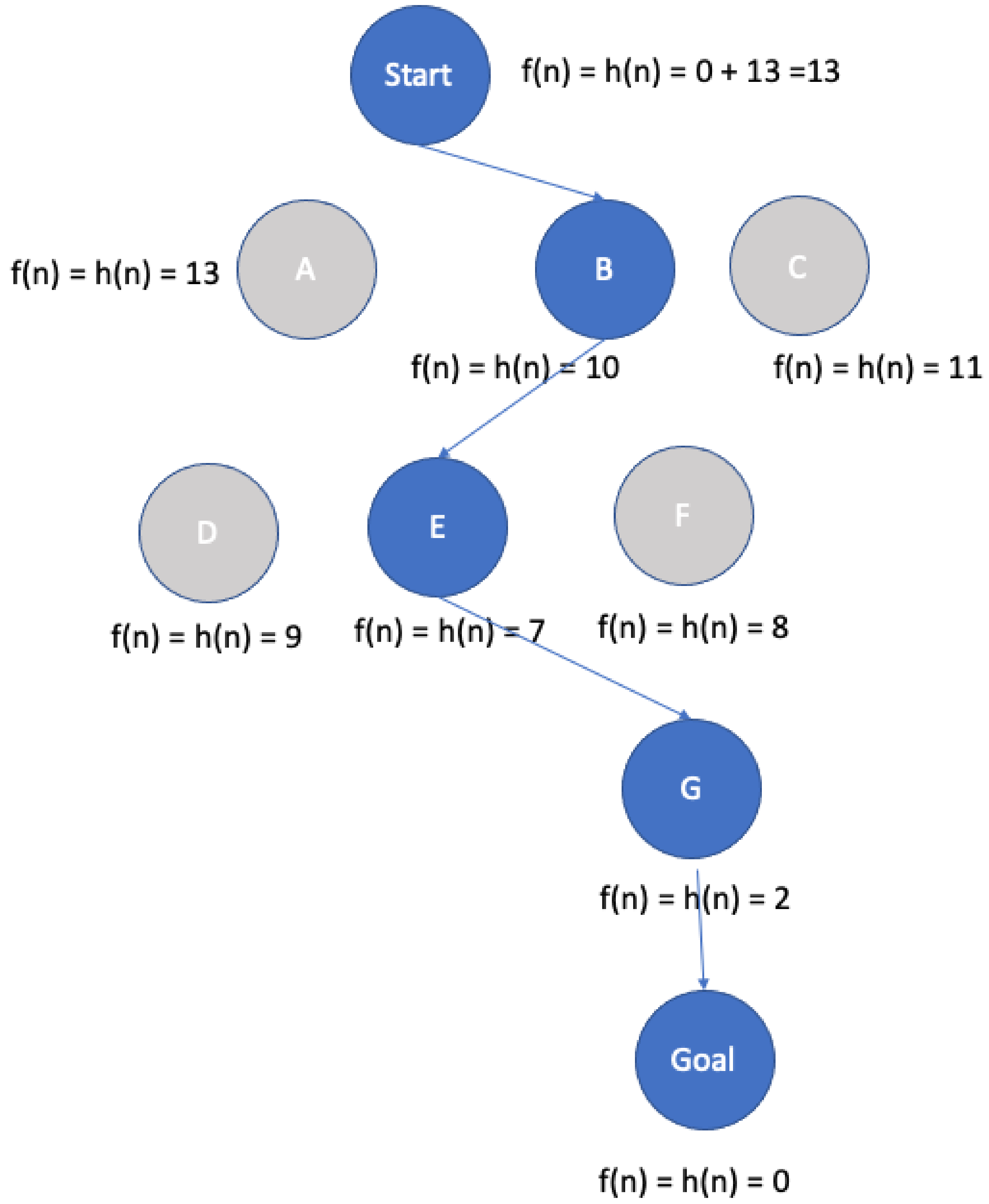
c. Explain why removing any one of the “fork” pieces makes the problem unsolvable.

A complete railway layout is only possible if every peg has a matching hole, meaning the total number of pegs and holes must be equal. If you remove a fork, the numbers of pegs and holes no longer match, so at least one connector will always remain open. This imbalance makes it impossible to form a closed layout

Graph-example (Greedy Algorithm)

$h(A)$ = 13	$h(B)$ = 10	$h(C)$ = 11	$h(D)$ = 9
$h(E)$ = 7	$h(F)$ =8	$h(G)$ = 2	$h(H)$ =3
$h(start)$ = 15	$h(Goal)$ = 0		



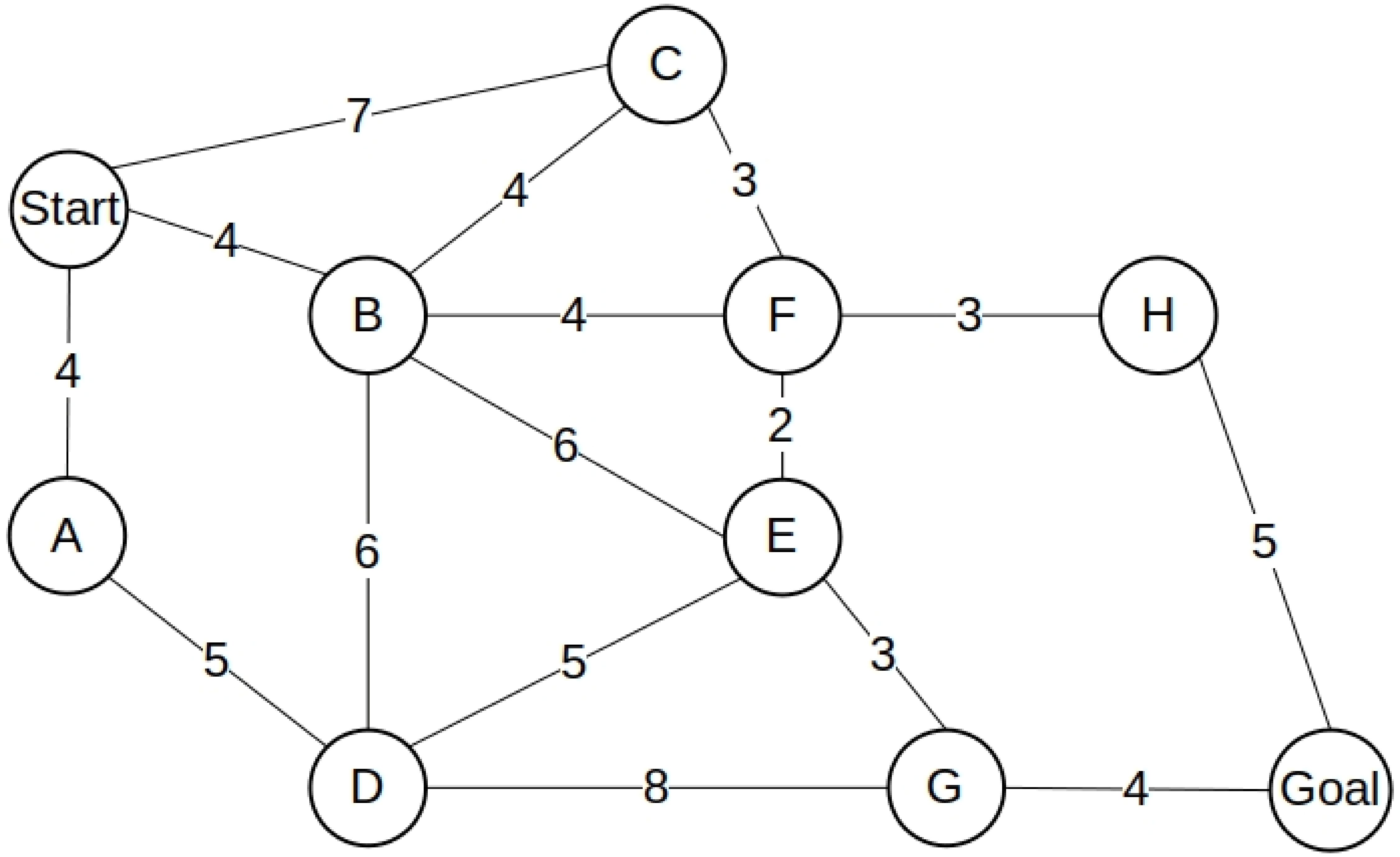


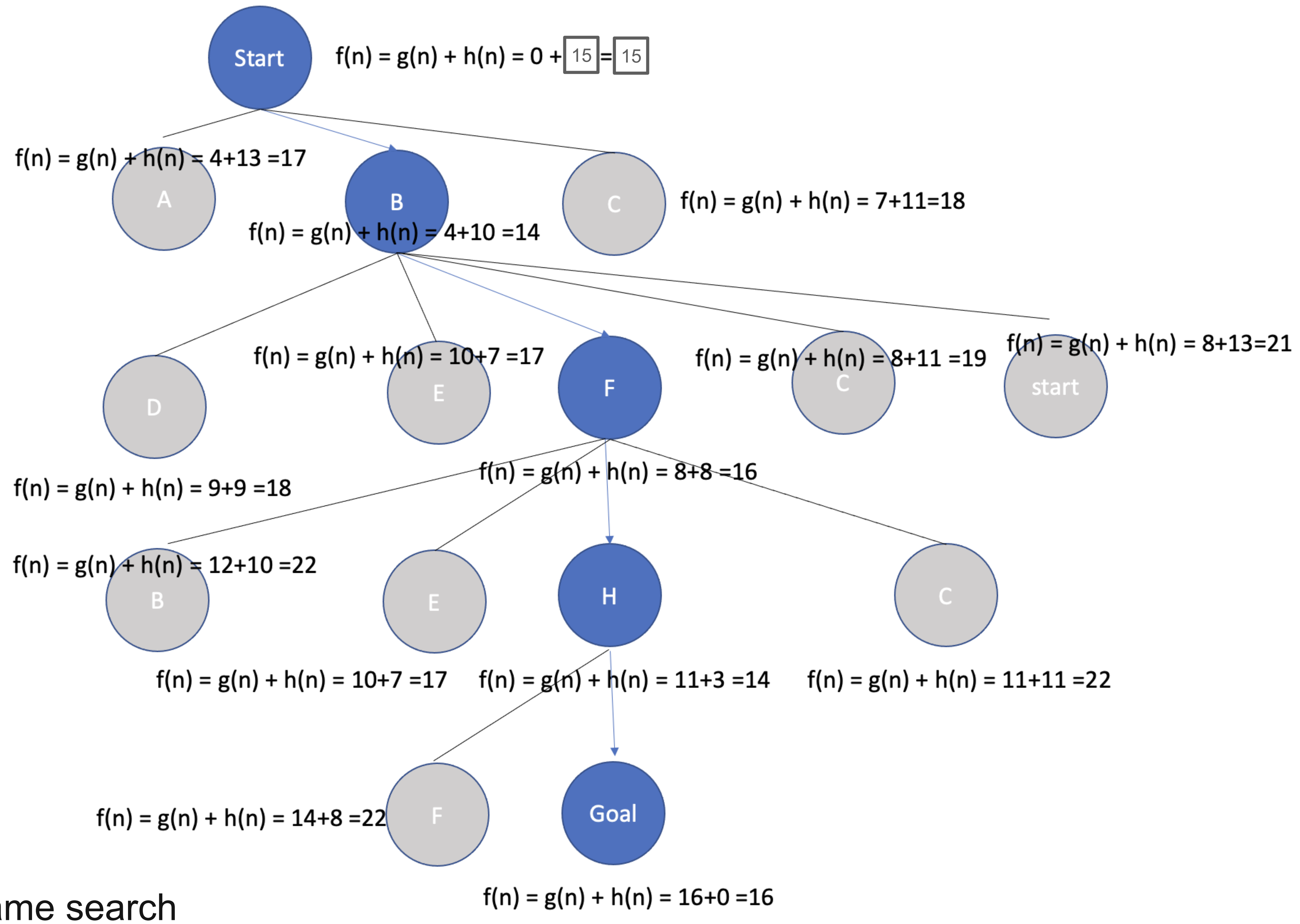
Example reference: https://www.baeldung.com/cs/dijkstra-vs-a-pathfinding

This path is not optimal!

Graph-example (A* Algorithm)

$h(A)$ = 13	$h(B)$ = 10	$h(C)$ = 11	$h(D)$ = 9
$h(E)$ = 7	$h(F)$ =8	$h(G)$ = 2	$h(H)$ =3
$h(start)$ = 15	$h(Goal)$ = 0		



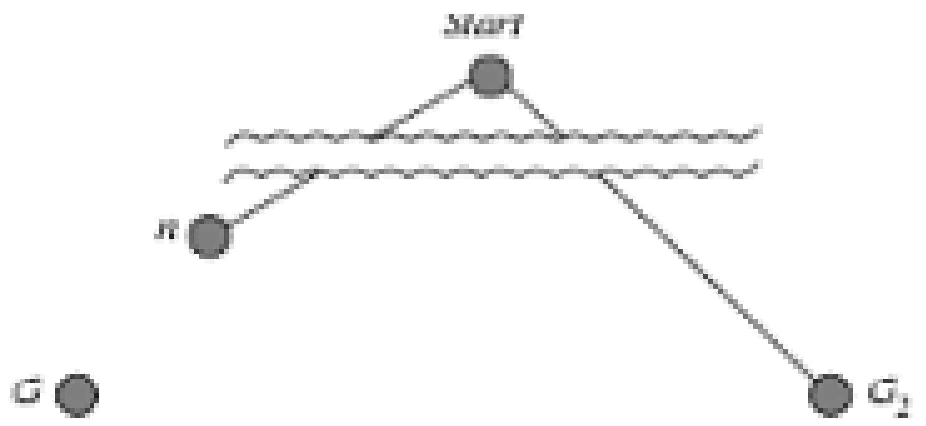


In the A* algorithm, once a node has been expanded, it should not be revisited during the same search

Example reference: https://www.baeldung.com/cs/dijkstra-vs-a-pathfinding

Admissible & Consistent

Admissible: $h(n) \leq h^*(n)$



$f(G_2)$	$= g(G_2)$	since $h(G_2)=0$
	$> g(G)=g(n)+h^*(n)$	since G_2 is suboptimal
	$>= f(n)$	since h is admissible

- G_2 is suboptimal, it is a goal state but with larger cost than the optimal goal state G
- n is an unexpanded node in the frontier and it is on the path to G

Consistent: $h(n) \leq c(n, n') + h(n')$

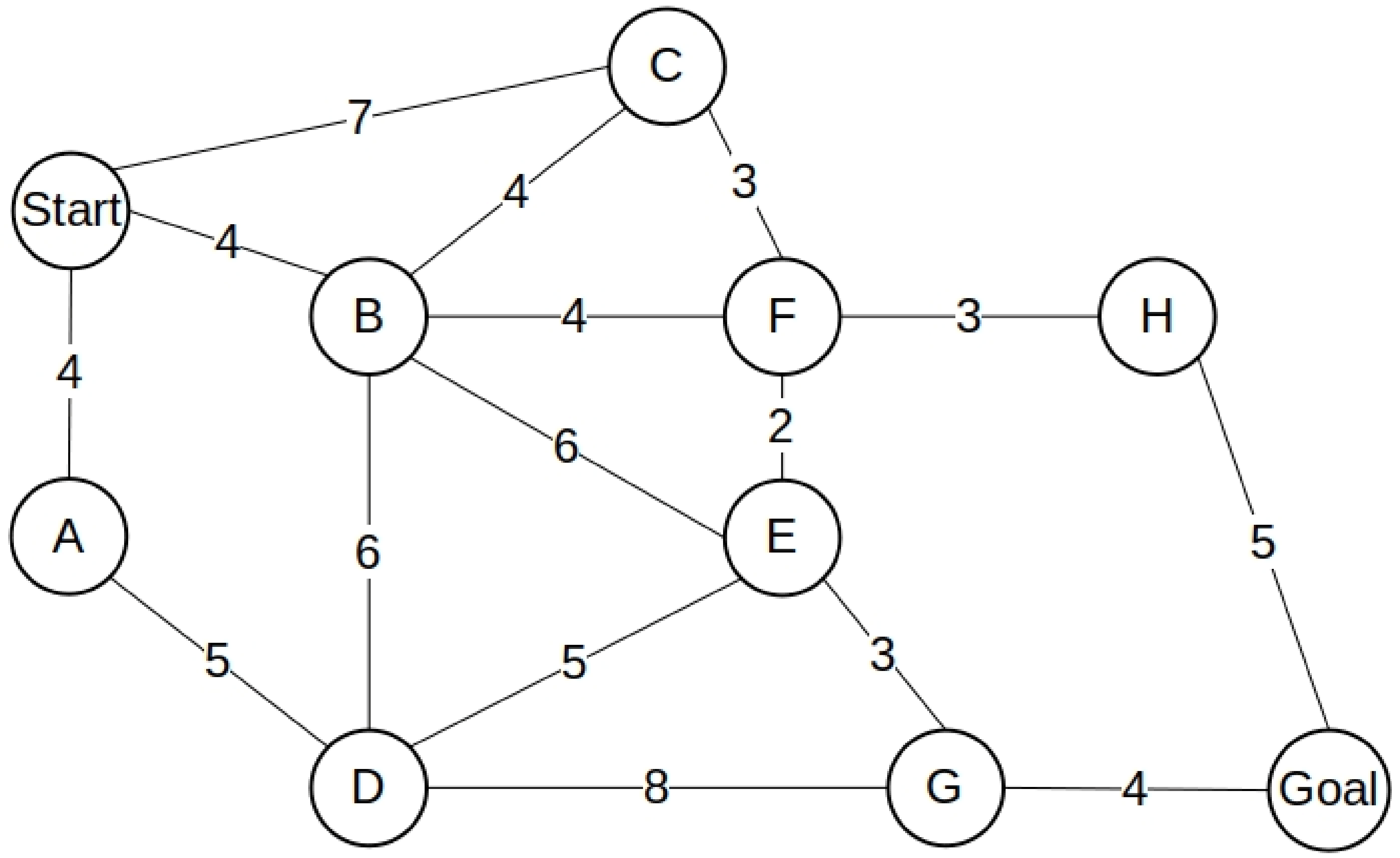
$$
\begin{aligned}
f(n') &= g(n') + h(n') \\
&= g(n) + c(n, n') + h(n') \\
&\geq g(n) + h(n) \\
&\geq f(n)
\end{aligned}
$$

- n′ is one child on n
- f(n) is non-decreasing along any path

Admissible $\underset{\longleftarrow}{\not\longrightarrow}$ Consistent

$$
\begin{aligned}
f(n) &\leq f(G) = g(n) + h^*(n) \\
g(n) + h(n) &\leq g(n) + h^*(n) \\
h(n) &\leq h^*(n)
\end{aligned}
$$

Admissible & Consistent example



$h(A)$ = 13	$h(B)$ = 10	$h(C)$ = 11	$h(D)$ = 9
$h(E)$ = 7	$h(F)$ =8	$h(G)$ = 2	$h(H)$ =3
$h(start)$ = 15	$h(Goal)$ = 0		

Admissible

$h(B) = 10 \leq 6 + 3 + 4 = h^*(B)$ $(B - E - G - Goal)$

$h(E) = 7 \leq 3 + 4 = h^*(E)$ $(E - G - Goal)$

Consistent

$B - F: h(B) = 10 \leq 4 + 8 = c(B,F) + h(F)$

Example for 8-puzzle (Heuristic functions)

1	8	2
	7	5
6	4	3

Initial state

1	2	3
8		4
7	6	5

Goal state

Admissible Heuristic

- Sum of Manhattan distances

0 1	2 8	1 2
	2 7	1 5
1 6	2 4	2 3

= 11

h(n) = 11≤ h*(n) = 14

Non-Admissible Heuristic (Nilsson's Sequence Score)

- h(n) = P(n) + 3 S(n); = 11 + 3·13

where P(n) = Sum of Manhattan distance,

S(n) = (center + 2 · successor) = (1 + 2 · 6) = 13

- Goal: [1,2], [2,3], [3,4], [4,5], [5,6], [6,7], [7,8]
- Current: [1,8], [8,2], [2,5], [5,3], [3,4], [4,6], [6,-]

1	8	2
	1 7	5
6	4	3

Due to h(n) = 50 ≥14


That's all Folks!